

Hands-on Lab: Create a DeepStream Pipeline with Jetson Nano

Objective

By the end of this lab, you will:

- Set up a **Jetson Nano** with NVIDIA DeepStream SDK
 - Deploy a real-time object detection pipeline from a camera feed
 - Publish detection metadata to an **MQTT broker** for integration with other systems
-

Step 1 — Prepare Your Jetson Nano

1. Download the latest **Jetson Nano Developer Kit SD Card Image** from NVIDIA's official site.
<https://developer.nvidia.com/embedded/jetson-nano-developer-kit>
 2. Flash the image onto a **32GB+ microSD card** using **balenaEtcher** or **Raspberry Pi Imager**.
 3. Insert the SD card into the Jetson Nano, connect:
 - USB keyboard & mouse
 - HDMI display
 - 5V 4A power supply
 - Ethernet cable or USB Wi-Fi
 4. Boot the device and complete the **initial Ubuntu setup**.
-

Step 2 — Update System and Install Dependencies

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y python3-pip v4l-utils
```

- **Why:** Ensures your system has the latest packages and tools needed for camera access.
-

Step 3 — Install NVIDIA DeepStream SDK

1. Download the **DeepStream for Jetson** installer:

```
wget https://developer.nvidia.com/deepstream_sdk_v6.3.0_jetson.tbz2
```

2. Extract and install:

```
tar -xvf deepstream_sdk_v6.3.0_jetson.tbz2
cd deepstream_sdk_v6.3.0_jetson
sudo ./install.sh
```

3. Set environment variables:

```
echo 'export PATH=$PATH:/opt/nvidia/deepstream/deepstream-6.3/bin' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/nvidia/deepstream/deepstream-6.3/lib' >> ~/.bashrc
source ~/.bashrc
```

Step 4 – Connect and Test Your Camera

- For USB cameras:

```
v4l2-ctl --list-devices
```

- Typically /dev/video0

- For CSI camera:

```
gst-launch-1.0 nvarguscamerasrc ! 'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! nvvidconv !
```

- Confirm that you get a live feed.

Step 5 – Configure the DeepStream Pipeline

1. Copy a sample DeepStream app:

```
cp -r /opt/nvidia/deepstream/deepstream/samples/configs/deepstream-app ~/deepstream-lab
cd ~/deepstream-lab
```

2. Edit `source1_usb_dec_infer_resnet_int8.txt` for USB camera:

```
[source0]
enable=1
type=3
camera-width=1280
camera-height=720
camera-fps-n=30
camera-fps-d=1
device=/dev/video0
```

3. Choose a pre-trained model (e.g., PeopleNet or TrafficCamNet) from `/opt/nvidia/deepstream/deepstream/samples/models`.

Step 6 – Enable MQTT Output for Metadata

1. Install the Mosquitto MQTT client:

```
sudo apt-get install -y mosquitto-clients
```

2. Edit `deepstream_app_config.txt` to include:

```
[message-broker]
enable=1
proto-lib=/opt/nvidia/deepstream/deepstream/lib/libnvds_mqtt_proto.so
conn-str=localhost;1883;deepstream/topic
topic=deepstream/detections
disable-tls=1
```

3. This configuration sends detection metadata as JSON to `deepstream/detections` topic on localhost.

Step 7 — Run the Pipeline

```
deepstream-app -c deepstream_app_config.txt
```

- You should see:
 - Real-time inference video feed with bounding boxes.
 - MQTT messages being published with detection details.
-

Step 8 — Monitor MQTT Messages

Open a second terminal and run:

```
mosquitto_sub -h localhost -t deepstream/detections
```

You'll see JSON output such as:

```
{
  "object": "person",
  "confidence": 0.95,
  "bbox": [120, 80, 200, 300],
  "timestamp": "2025-08-07T15:25:32Z"
}
```

Step 9 — Optional: Send to Remote MQTT Broker

- Change `conn-str` in config to:

```
conn-str=broker.hivemq.com;1883;deepstream/topic
```

- Now your detections can be consumed by any IoT dashboard or cloud service.

Step 10 — Clean Up & Shutdown

```
sudo poweroff
```

- Safely disconnect power after shutdown.

End Result

You now have:

- A **real-time object detection pipeline** running on a Jetson Nano with DeepStream.
 - **Live AI inference** from a camera feed.
 - **Detection events** streaming via MQTT for integration into dashboards or automation systems.
-